



UNIVERSIDAD

Taller de diferenciación y ecuaciones diferenciales

Estudiantes

Lenin Enrique García Caraballo

David Orlando Riaño Morales

Docente

Alexander Pérez Ruiz

Escuela Colombiana de Ingeniería Julio Garavito

Métodos computacionales en ingeniería – MCEI

2026-2

3 de octubre de 2026

1 PRIMER EJERCICIO

1.1 PARTE 1

Las gráficas obtenidas utilizando Octave (con el código llamado `cinematica_diferencial.m`)

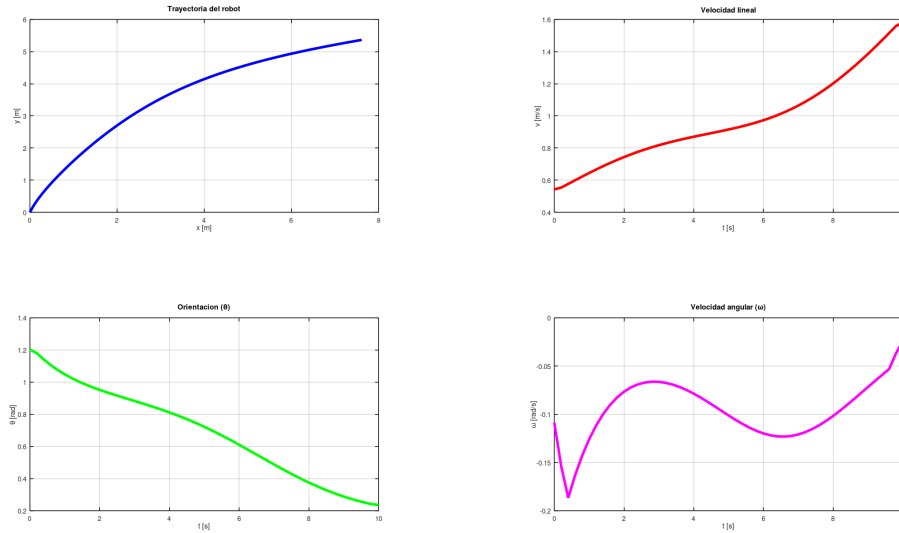


Figura 1: Gráficas de y vs x , v vs t , θ vs t y ω vs t en Octave

1.2 PARTE 2

Las gráficas obtenidas utilizando la solución vectorizada de NumPy en Python (con el código llamado `parte2_taller_robot.ipynb`):

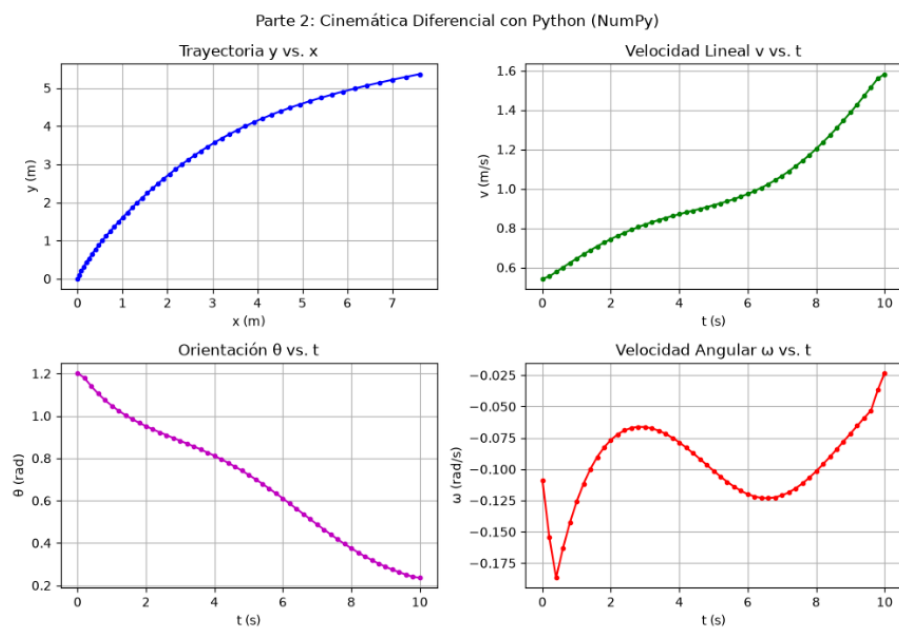


Figura 2: Gráficas de y vs x , v vs t , θ vs t y ω vs t

En Octave, fue necesario inicializar vectores con ceros e implementar un ciclo for iterando desde $i = 2$ hasta $N-1$ para aplicar la fórmula de diferencia central punto por punto.

En Python, textbfNumPy usa `np.gradient(x, t)` para aplicar internamente la ecuación de diferencia central. La biblioteca ejecuta esta misma aproximación central sobre todos los datos interiores de forma

automática y vectorizada, quitando la necesidad de bucles **for** tal como se hace en Octave.

En Octave en el primer extremo de los datos, se hace diferenciación hacia adelante debido a que no hay un dato anterior al primer dato, y que en el último extremo debe hacer diferenciación hacia atrás porque no hay un dato siguiente al último dato.

Además **np.gradient** identifica automáticamente los límites del arreglo y aplica aproximaciones unilaterales en los extremos para garantizar que el vector resultante conserve exactamente el mismo número de elementos que el vector original.

1.3 PARTE 3

El código en C++ con GSL usado es **cinematica_diferencial.cpp**:

t(s)	v(m/s)	theta(rad)	w(rad/s)
0.00	0.5426	1.2017	-0.1089
0.20	0.5539	1.1799	-0.1541
0.40	0.5768	1.1401	-0.1867
0.60	0.5998	1.1053	-0.1630
0.80	0.6225	1.0748	-0.1429
1.00	0.6447	1.0481	-0.1259
1.20	0.6663	1.0245	-0.1117
1.40	0.6871	1.0034	-0.0999
1.60	0.7070	0.9845	-0.0903
1.80	0.7259	0.9673	-0.0826
2.00	0.7437	0.9515	-0.0766
2.20	0.7606	0.9367	-0.0721
2.40	0.7764	0.9226	-0.0690
2.60	0.7911	0.9091	-0.0671
2.80	0.8049	0.8958	-0.0663
3.00	0.8177	0.8826	-0.0664
3.20	0.8296	0.8692	-0.0674
3.40	0.8407	0.8556	-0.0693
3.60	0.8511	0.8415	-0.0718
3.80	0.8610	0.8268	-0.0750
4.00	0.8704	0.8115	-0.0787
4.20	0.8794	0.7954	-0.0828
4.40	0.8883	0.7784	-0.0872
4.60	0.8971	0.7605	-0.0918
4.80	0.9061	0.7417	-0.0966
5.00	0.9154	0.7219	-0.1013
5.20	0.9252	0.7012	-0.1058
5.40	0.9357	0.6795	-0.1101
5.60	0.9471	0.6571	-0.1140
5.80	0.9595	0.6340	-0.1173
6.00	0.9731	0.6102	-0.1199
6.20	0.9881	0.5860	-0.1218
6.40	1.0046	0.5615	-0.1229
6.60	1.0227	0.5368	-0.1232
6.80	1.0426	0.5122	-0.1225
7.00	1.0644	0.4878	-0.1210
7.20	1.0881	0.4638	-0.1185
7.40	1.1137	0.4404	-0.1153
7.60	1.1413	0.4177	-0.1114
7.80	1.1708	0.3958	-0.1068
8.00	1.2023	0.3750	-0.1017
8.20	1.2357	0.3552	-0.0961
8.40	1.2709	0.3365	-0.0902
8.60	1.3079	0.3191	-0.0841
8.80	1.3464	0.3029	-0.0779
9.00	1.3865	0.2879	-0.0716
9.20	1.4280	0.2742	-0.0654
9.40	1.4707	0.2617	-0.0593
9.60	1.5145	0.2505	-0.0533
9.80	1.5593	0.2404	-0.0369
10.00	1.5820	0.2357	-0.0234

Figura 3: Resultados de v(m/s), theta(rad) y w(rad/s)

La función "gsl_deriv_central" de GSL se utiliza para calcular la derivada de una función que está definida

de forma continua. Esto significa que el programa puede evaluar la función en cualquier valor de x que necesite. A partir de esas evaluaciones, el método utiliza puntos cercanos al valor donde se quiere calcular la derivada y obtiene una aproximación de su pendiente.

Cuando se trabaja con datos discretos, como los datos de la trayectoria del robot, solo se conocen los valores registrados en determinados instantes de tiempo. Por ejemplo, si el robot fue medido en $t = 1$ s y $t = 2$ s, no se conoce su posición en $t = 1,5$ s. Por esta razón, no se puede utilizar la función de la misma manera que con una función continua.

En este caso, lo más adecuado es utilizar métodos de diferencias sobre los datos disponibles. Para los puntos intermedios se pueden utilizar diferencias centrales, tomando el dato anterior y el siguiente para aproximar la derivada. En los extremos se utilizan diferencias hacia adelante o hacia atrás, ya que no existen datos a ambos lados.

1.4 Comparación de resultados

Criterio	Octave	Python	C/C++ con GSL
Carga de datos	Sencilla con <code>csvread</code> , pero requiere saltar encabezados manualmente.	Sencilla y compacta usando NumPy (<code>np.loadtxt</code>).	Laboriosa. Implica el manejo explícito de flujos de archivos (<code>ifstream</code>) y separación manual por comas (<code>stringstream</code>).
Cálculo de $\dot{x}$ y $\dot{y}$	Es manual, porque requiere inicializar vectores en cero y usar un ciclo <code>for</code> para aplicar la fórmula de diferencia central.	Automático, porque la función <code>np.gradient</code> aplica diferencias centrales sin necesidad de ciclos <code>for</code> .	Complejo, porque exige declarar los vectores, definir su tamaño de memoria e iterar punto a punto con diferencias centrales y unilaterales.
Cálculo de v	Directo mediante operaciones matriciales $v = \sqrt{v_x.^2 + v_y.^2}$.	Directo y vectorizado con <code>np.sqrt</code> sobre arreglos de NumPy.	Manual, porque obliga recorrer los arreglos iterando dato por dato usando <code>std::sqrt</code> .
Cálculo de θ	Directo usando <code>atan2</code> vectorizado sobre las matrices.	Directo, vectorizado con <code>np.arctan2</code> .	Manual, porque requiere iterar sobre los elementos usando <code>std::atan2</code> .
Cálculo de ω	Es manual, porque requiere la función <code>unwrap</code> y luego aplicar la diferencia central con un ciclo <code>for</code> , al igual que las velocidades.	Es automático, tras usar <code>np.unwrap</code> , <code>np.gradient</code> calcula la derivada manejando automáticamente los extremos.	Muy complejo, porque se debe implementar la lógica de desdoblamiento (<code>unwrap</code>) desde cero antes de derivar manualmente con ciclos.
Manejo de arreglos	Matrices dinámicas con indexación que inicia en 1 (1-based).	Arreglos vectorizados (<code>ndarrays</code>) altamente optimizados. Indexación desde 0.	Contenedores <code>std::vector</code> que requieren una gestión estricta del tamaño de memoria. Indexación
Facilidad de implementación	Media, porque la sintaxis es matemática, pero la derivación requiere ciclos explícitos.	Alta, porque las funciones de alto nivel reducen la longitud del código y la complejidad.	Poca facilidad, porque requiere un control explícito de los tipos de datos y la gestión de memoria.
Control sobre el algoritmo	Alto control, debido a que se define la ecuación analítica directamente en el código.	Poco control, porque <code>np.gradient</code> actúa como una caja negra que encapsula la lógica.	Se tiene el control total de cada operación aritmética y salto de memoria.
Tiempo de ejecución (Segundos)	0,00654 (Es el lenguaje más lento para esta tarea)	0,000817 (El segundo más lento)	0,000196751 (C++ al ser de bajo nivel, permite ejecutar las operaciones más rápidamente, pero siendo más complejo que Python y Octave)

Figura 4: Tabla comparativa entre Octave, Python y C++ con GSL

Sobre el método numérico: Para obtener las velocidades del robot a partir de datos discretos, es necesario seguir un orden. Primero se calculan las velocidades en los ejes x y y , es decir, $\dot{x}$ y $\dot{y}$. Con ellas se calcula la velocidad lineal v y la orientación θ mediante `atan2`. Después, se aplica el `unwrap` para eliminar los saltos del ángulo y , finalmente, se deriva θ para obtener la velocidad angular ω .

Durante este proceso, el ruido o los errores presentes en las posiciones pueden afectar las velocidades calculadas. Este efecto se hace más grande cuando el paso de tiempo h es muy pequeño. Además, como ω se obtiene después de calcular y derivar la orientación, es más sensible a los errores que la velocidad lineal v .

Sobre el entorno computacional: El lenguaje utilizado también influye en la forma de implementar el método. Python, especialmente utilizando NumPy, permite realizar los cálculos de manera sencilla y con pocas líneas de código, también es el mismo caso con Octave. Funciones como `np.gradient` permiten calcular derivadas sin tener que programar manualmente cada operación. Pero, esto hace que algunas cosas no se puedan configurar porque parte del proceso queda dentro de la función.

En C++, es necesario controlar de forma más directa aspectos como los arreglos, los índices y las operaciones utilizadas para calcular las derivadas. Esto hace que el código sea más largo y requiera mayor cuidado, pero también permite entender y controlar de qué forma se hacen los cálculos.

Python y Octave facilitan la implementación y permiten obtener resultados rápidamente, mientras que C++ permite tener un mayor control sobre el procedimiento.

1.5 Preguntas de análisis

- ¿Qué diferencia existe entre calcular la velocidad lineal a partir de $\dot{x}$, $\dot{y}$ y calcularla directamente desde diferencias de posición?

La velocidad lineal se puede calcular de dos maneras. La primera consiste en calcular por separado cómo cambia la posición del robot en cada eje, es decir, obtener $\dot{x}$ y $\dot{y}$. Luego, estas dos componentes se combinan para obtener la velocidad total (calculando la magnitud):

$$v = \sqrt{\dot{x}^2 + \dot{y}^2}$$

La ventaja de este método es que, además de conocer qué tan rápido se mueve el robot, también se conserva la información sobre la dirección en la que se está moviendo. Por ejemplo, a partir de $\dot{x}$ y $\dot{y}$ se puede calcular la orientación mediante $\theta = \text{atan2}(\dot{y}, \dot{x})$.

La segunda opción es calcular la distancia entre dos posiciones consecutivas y dividirla entre el tiempo transcurrido. Esto permite saber qué tan rápido se desplazó el robot entre esos dos puntos, pero no indica en qué dirección se estaba moviendo.

- ¿Por qué un error pequeño en $\dot{x}$, $\dot{y}$ puede producir un efecto mayor al calcular ω ?

Un pequeño error en $\dot{x}$ o $\dot{y}$ puede producir un error mayor en ω porque la velocidad angular se calcula en varios pasos. Primero, las componentes de la velocidad se utilizan para calcular la orientación del robot con $\theta = \text{atan2}(\dot{y}, \dot{x})$. Por lo tanto, cualquier pequeña variación en $\dot{x}$ o $\dot{y}$ puede producir una variación en el ángulo calculado.

Este efecto se hace más importante cuando el robot se mueve lentamente, ya que sus velocidades $\dot{x}$ y $\dot{y}$ son pequeñas y el ruido representa una parte más significativa de los valores medidos. Además, para obtener la velocidad angular se debe calcular nuevamente una derivada de la orientación:

$$\omega = \frac{d\theta}{dt}$$

La derivación puede hacer más visibles las pequeñas variaciones presentes en los datos. Por esta razón, un pequeño ruido en las velocidades puede generar cambios en θ y valores más variables de ω .

- ¿Qué ocurre si se reduce h manteniendo un nivel de ruido fijo en las posiciones?

Si se reduce el paso de tiempo h , pero se mantiene el mismo nivel de ruido en las posiciones, el ruido tiene un efecto mayor sobre las derivadas calculadas. Esto ocurre porque en las fórmulas de diferencias, se divide el cambio de posición entre un intervalo de tiempo h . Al hacer h más pequeño, el ruido tiene mayor influencia.

Si dos posiciones tienen una pequeña diferencia debido al ruido, al dividir esa diferencia entre un intervalo de tiempo grande su efecto será pequeño. Pero si el intervalo de tiempo es muy pequeño, la misma diferencia se divide entre un número menor y produce un error mayor.

- ¿Por qué ω requiere especial cuidado con el desenvolvimiento de θ ?

La velocidad angular ω requiere especial cuidado con el desenvolvimiento de θ porque el ángulo obtenido con atan2 está limitado a un rango entre $-\pi$ y π . Esto significa que cuando el robot gira y supera ese límite, el valor del ángulo puede pasar, por ejemplo, de 3,14 rad a $-3,14$ rad, aunque el robot haya continuado girando en la misma dirección.

Este cambio no representa un giro real, sino un salto producido por la forma en que atan2 representa los ángulos. Esto genera un problema cuando se calcula la velocidad angular, ya que esta se obtiene a través de la derivada del ángulo.

Si el salto no se corrige, la derivada interpreta ese cambio como si el robot hubiera girado 2π en un intervalo de tiempo muy pequeño. Esto genera un valor de ω muy grande que no corresponde al movimiento real.

Para solucionarlo, antes de calcular la velocidad angular es necesario aplicar el desenrollamiento angular (unwrap). Este proceso elimina los saltos de 2π y permite que θ cambie de forma continua.

1.6 Reflexión

- Sin consultar el código, reconstruya el procedimiento matemático utilizado para pasar de (x, y, t) a v y ω . Explique qué derivadas deben calcularse, en qué orden y dónde puede introducirse error numérico.

Primero, se calculan las velocidades en cada eje a partir de las posiciones. Para ello se obtiene la derivada de x respecto al tiempo, $\dot{x}$, y la derivada de y respecto al tiempo, $\dot{y}$. Estas representan qué tan rápido cambia la posición del robot en cada dirección.

Luego, se utilizan estas dos componentes para calcular la velocidad lineal del robot. Como $\dot{x}$ y $\dot{y}$ forman un vector de velocidad, su magnitud se obtiene con:

$$v = \sqrt{\dot{x}^2 + \dot{y}^2}$$

Después se calcula la orientación del movimiento. Para ello se utilizan las dos componentes de la velocidad en la función atan2:

$$\theta = \text{atan2}(\dot{y}, \dot{x})$$

Este ángulo indica hacia qué dirección se está moviendo el robot. Como atan2 puede producir saltos de 2π cuando el ángulo pasa de un lado al otro de su rango, se debe usar de desenrollamiento angular (unwrap) para evitar estos saltos.

Por último, se calcula la velocidad angular a partir de la variación de ese ángulo con respecto al tiempo:

$$\omega = \frac{d\theta}{dt}$$

El orden completo del procedimiento es:

$$(x, y, t), (\dot{x}, \dot{y}), v$$

y, utilizando las mismas componentes de velocidad:

$$(\dot{x}, \dot{y}), \theta, \text{unwrap}(\theta), \omega$$

El error numérico puede aparecer al calcular las derivadas. Los datos de posición pueden contener ruido y, al calcular $\dot{x}$ y $\dot{y}$, ese ruido se hace más visible. Además, el tamaño del intervalo de tiempo h influye en el resultado, si es pequeño y existe ruido en las mediciones, el error puede aumentar.

Estos errores también pueden afectar el cálculo de θ , ya que este depende de $\dot{x}$ y $\dot{y}$. Al calcular otra derivada para obtener ω , las variaciones de θ pueden aumentar. Por esta razón, la velocidad angular suele ser más sensible al ruido que la velocidad lineal.

Si recibiera datos reales de un robot con otra frecuencia de muestreo y ruido de sensores, ¿qué decisiones del taller conservaría, cuáles modificaría y qué criterio utilizaría para seleccionar el método de diferenciación?

Mantendría la estructura general utilizada en el taller, primero calcularía las velocidades en los ejes x y y , luego obtendría la velocidad lineal a partir de estas componentes y calcularía la orientación con atan2. También mantendría el desenrollamiento angular (unwrap) antes de calcular la velocidad angular, ya que es necesario evitar los saltos de 2π en el ángulo.

Lo que modificaría sería la forma de calcular las derivadas. En el taller se utilizan diferencias directamente sobre los datos, pero con datos reales los sensores pueden tener ruido. Como la diferenciación puede hacer que ese ruido tenga más influencia, primero se debería aplicar un filtrado.

La frecuencia de muestreo también sería importante. Si el robot tiene un paso de tiempo h muy pequeño, se tienen más mediciones, pero el ruido puede tener un efecto mayor sobre las derivadas. Si h es muy grande, se pierde información del movimiento. Hay que tener un equilibrio entre la frecuencia de muestreo y la cantidad de ruido.

2 SEGUNDO EJERCICIO

Se tiene el siguiente sistema masa-resorte-amortiguador sin una entrada, y con salida x . Se quiere obtener la salida si la posición inicial es 1 metro, y su velocidad inicial es de 0.

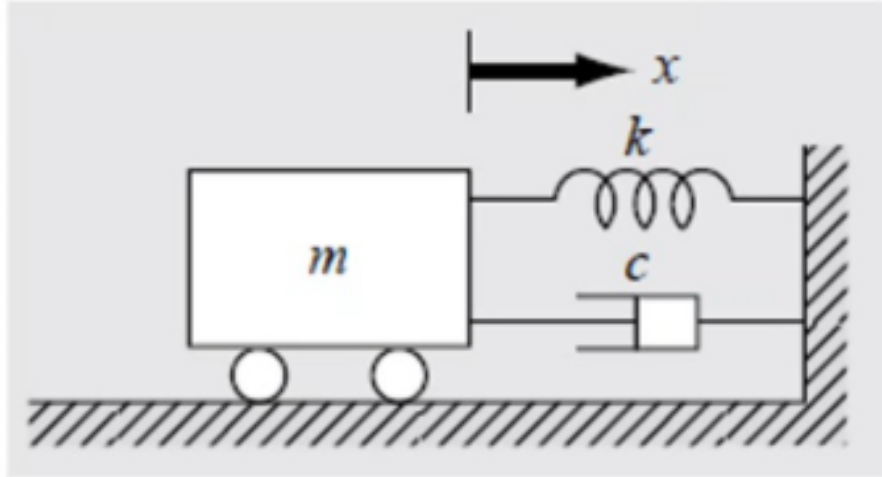


Figura 5: Sistema masa resorte amortiguador

La ecuación diferencial es la siguiente:

$$m \frac{d^2 x}{dt^2} + c \frac{dx}{dt} + kx = 0 \quad (1)$$

Al plantear sus variables de estado, se obtiene lo siguiente: $x_1 = x$ $x_2 = \frac{dx}{dt}$

$$\frac{dx_1}{dt} = x_2$$

Por lo tanto, el sistema de primer orden es:

$$\frac{dx_2}{dt} = -\frac{k}{m}x_1 - \frac{c}{m}x_2 \quad (2)$$

Se resolvió en Python (llamado **masa_resorte_ejercicio.ipynb**) el sistema teniendo en cuenta las condiciones iniciales y los valores para los coeficientes: masa de $m = 20$ kg, coeficiente de amortiguamiento $c = 5$, $c = 40$ y $c = 200$, y constante de elasticidad $k = 20$ N/m. Esto con la ayuda de las funciones básicas de Python y el uso de arreglos de la librería NumPy.

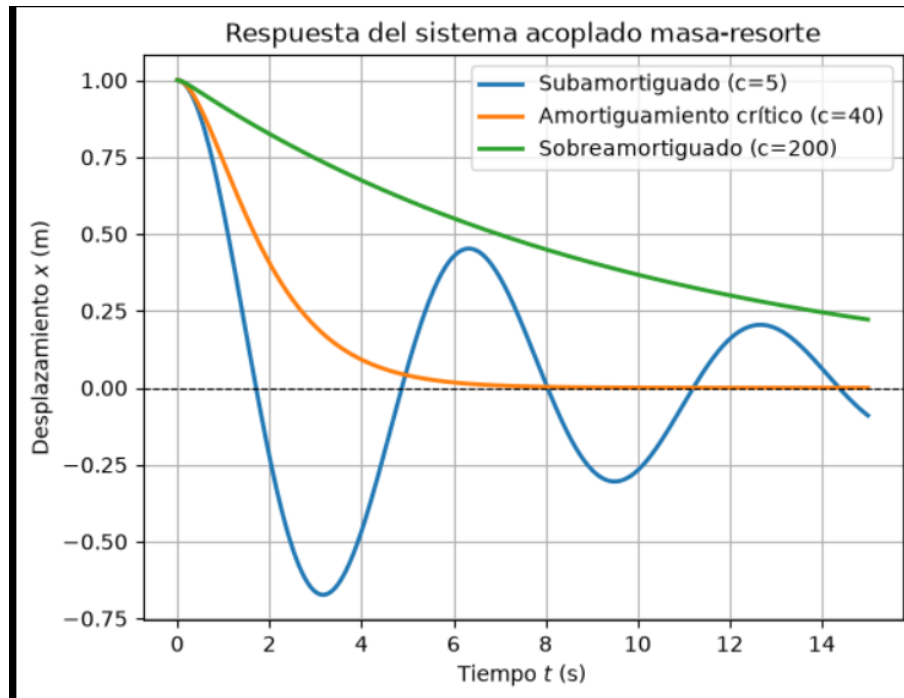


Figura 6: Salidas del sistema con coeficientes de amortiguamiento $c = 5$, $c = 40$ y $c = 200$ en Python

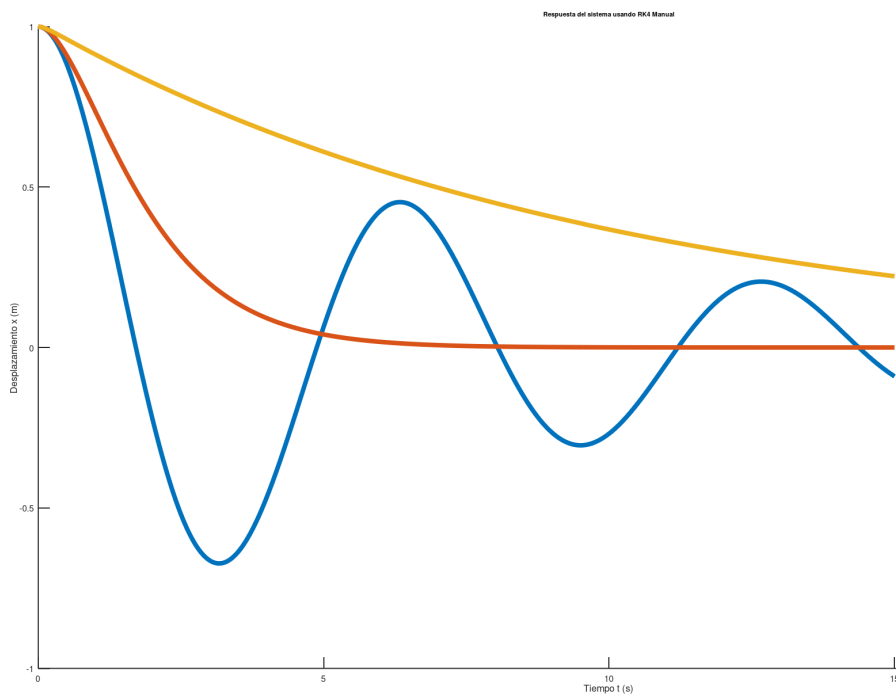


Figura 7: Salidas del sistema con coeficientes de amortiguamiento $c = 5$, $c = 40$ y $c = 200$ en Octave

Para Octave, se realizó el mismo procedimiento de Python usando el código `ecuacion_diferencial.m`.